Phase II Projects

Now that you have a crime story generator, we will make your system interactive. This means the user will assume the role of a character in the story world. The user can be the detective (mystery solver), or a companion to the detective. The story that plays out will be one generated by your Phase I story generator. Consistent with the discussion of AI Interactive Storytelling, the user should be able to perform actions that are part of the story (constituent), or not explicitly part of the story (consistent). The user should be able to perform actions that are not part of the story and derail the story (exceptions). Examples include making it impossible to uncover a clue and/or solve the crime (for example, destroying a clue or doing something that makes it impossible to uncover a clue like permanently locking a door, killing a witness, etc.), but also uncovering clues earlier than expected or in a different order that breaks the flow of the story.

The system will be a text game, meaning that all interactions between the user and the system will be through text input and output. Descriptions of the user's location and state will be presented as text. The user will be able to describe their actions as text and see how the world changes. The user's interactions should be open-ended, meaning they can type anything in, though you may want to constrain the length of the user's commands to, for example, length of 5. User interactions should not be from a menu of options. You will thus need to determine how the world changes and whether the user's actions are constituent, consistent, or exceptional.

You will select one of the project templates, which lay out some high-level guidelines to be met.

The templates are based off research literature on AI interactive storytelling, but the research literature we draw from does not consider crime mystery stories. Thus, even when templates are based heavily on specific research papers, there will be elements for which some novel innovation will be required to fit the technique to the assignment. The details of how you mesh the requirements of the template, the background literature, and the assignment requirements will be the crux of your initial team proposal, the first of several milestones.

Milestone 1: Proposal Video

In this milestone, you will create a presentation as a video recording that indicates the template you will use, and your rough sketch of how your system will use your story generator from Phase I and handle user actions. The most straightforward approach will be a video made of slide presentations with voice-over.

Milestone 2: Cross-Team Proposal Assessments

You will review the proposal videos of other teams and provide constructive assessment of what elements from lecture might improve the proposal. Teams will not be required to act upon these and indeed will never see other teams' assessments. The goal of this is to reflect on what other teams are doing and to reflect on lecture material.

Milestone 3: Final Deliverables

There are two deliverables:

(a) A final presentation and demonstration **video**. Your final presentation will explain how your system works, and demonstrate your system in action. You are not required to record your system running in real-time, but it should provide examples of the outputs of one or more successful runs and one or more unsuccessful runs.
(b) Your **codebase**, fully documented so we can read through the code to see how it works. The code should be playable.

Milestone 4: Cross-Team Proposal Assessments

You will review the final videos of other teams and provide constructive assessment of what elements from lecture might have improved the final design. Teams will not be required to act upon these and indeed will never see other teams' assessments. The goal of this is to reflect on what other teams are doing and to reflect on lecture material.

Milestone 5: Self- and Team-Assessment

You will provide a brief document reporting on your role on the team project and your assessment of your performance as a team member. You will also report on the roles your teammates took and your assessment of their performances as team members. The purpose of this document is to help the instructional team determine whether the entire team should receive the same grade.

Grading rubrics for all deliverables will be provided separately

Playability Metrics

The length of the interactive experience is dictated by the length of the stories generated by your Phase I generator. It is thus expected that at least 15 major events will unfold, though it may be the case that not all of the major events are initiated by the user. The user should not be able to shortcut straight to the end of the story (i.e., solve the crime immediately without uncovering clues). The user may do more actions because they may perform

extraneous actions like moving around the world or trying actions that fail to uncover clues (e.g., interviewing witnesses in the wrong order).

Build a Text Game Engine

A text game engine prints output text and receives text input. Canonical examples include Zork and Anchorhead, which have online playable versions. At the heart, any game engine, including text game engines, maintain a global world state. This is typically a data structure, or collection of data structures that contain information about the layout of the world—the "rooms" that the player can visit—objects to be found in rooms, non-player characters, and any relevant state information about the player including inventory. Traditionally, the world is a discrete collection of rooms (though a room can be described as being outdoors) that can be navigated between with actions such as "north", "south", northwest", etc., but also sometimes through special actions like "down" or "climb through window", etc. Thus, the game world maintains a graph of rooms and arcs with information about how to traverse. Objects and characters, including the player's character hold information about the location they are in. Objects and characters also maintain pertinent state information (e.g. an object can be in the state of being "damaged", a character can have a broken finger. This state information typically constrains what actions can be taken. For example: a damaged toaster cannot be used to make toast; a character with a broken finger cannot pick a lock. Actions may cause these state variables to change value, such as dropping a toaster damages it, but the repair action (which may require the player to have certain tools in inventory) can remove the damage. State variables do not strictly need to be binary.

Users enter text to describe their actions. These statements must be interpreted. In classical text games, there is a large but finite set of verbs that may or may not take parameters ("pick up stone", "go west"). Actions can only be executed if certain rule conditions hold. For example: to pick up a stone there must be a stone present; to go west there must be a transition in the location graph labeled as "west". The manual creation of the verbs and the interpreter to identify which verbs the user is trying to execute can be considered a limitation of most text games. It may be possible, even desirable, to have an LLM interpret the user's actions. If so, then one must either figure out how to map the user's input into the set of actions that the game engine can recognize and execute, or determine on the fly how the user's actions will change the state of the game world (and whether this derails the story, which is the topic of some of the project templates).

Generally speaking, your story generator likely does not use a fixed set of actions or events based on a finite set of verbs. Your game engine will possibly need to do some generation

of actions and how those actions affect state variables. Your game engine may need to rely on the story generator to determine what the relevant state variables should be and how they must change when actions are performed. This information should be provided in your proposal and final videos.

Story Generation

A story should play out in the text world, in the sense that events occur that are initiated by non-player characters or by the user. Should the correct sequence of events be performed, then there is a successful conclusion. Thus, one can see the story as analogous to a finite state machine in which the plot points of the story are enacted in an acceptable order.

If your story generator generates plot points and then elaborates them into fluent, reader-friendly text for final output, your story generator may be modified to provide the plot point specifications necessary for interactive execution in your game engine.

World Generation

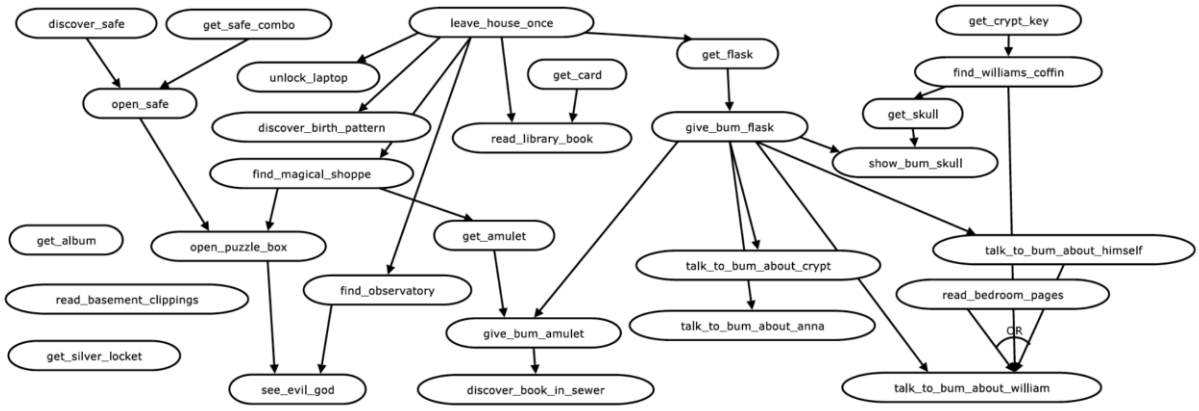
It is very likely that your story generator did not have an explicit map of locations that would be helpful for the text game engine. World generation is a process of analyzing the story output of the story generator to identify the locations where events take place. If events happen in unspecified locations, it may be necessary to elaborate on the locations of events after generation is complete but before interactive execution begins. (And some events do not require locations or can happen in any location, like “whistle a tune”). Once the locations are known, there must be graph connectivity between locations. Some locations may naturally be adjacent to each other (event 1 takes place in a bedroom and event 2 takes place in a living room, which can be adjacent to the bedroom). For other locations it may make sense for there to be intermediary locations (event 1 takes place in a bedroom and event 2 takes place in a restaurant). If adjacent events should not be in adjacent locations, from a commonsense perspective, then intermediate locations should be generated and connected.

There is no requirement for the size of the world. It should contain the minimal number of locations to support all the events of the story, plus however many additional locations necessary so that the adjacent locations do not violate commonsense.

Project Templates

1. Search-Based Drama Management

Search Based Drama Management is analogous to “playing chess” between the user and the Drama Manager. A plot graph captures the temporal ordering requirements between plot points. Consider this example for the text adventure game, Anchorhead:



The plot graph tells us that certain plot points, such as “discover safe” and “get safe combo” must happen before the safe can be opened. It is an expression of the underlying rules of the game (must be at the location of the safe and must have found the safe combination before the safe combination can be entered into the safe) but may also serve as execution logic (open safe only executes successfully if its predecessors have all been executed successfully).

The plot graph thus imposes constraints on possible total orderings of events. However, the plot graph does not tell us if any possible total orderings are better than others. If there is an evaluation function that can score the “goodness” of a totally ordered sequence, we still wouldn’t know if the user will choose that total ordering.

In Search Based Drama Management, we see Drama Management as a two-player game: the user can perform a move, then the drama manager can perform a move. The player’s moves are executing plot points, and which plot points are possible are constrained by the plot graph. The DM’s moves are to:

- cause a plot point to happen
- permanently deny a plot point (make it impossible for that plot point to occur)
- temporarily deny a plot point (make it impossible for a plot point to occur for a finite period of time)

- give the player a hint (make something glow, instruct an NPC to say something to the player, etc.)

The player makes a move, changing the state of the world. For each possible state that the world can transition to, the DM makes a move. For every state the world can be in after a player and a DM move, the player can make a new move. And so on. This can be done in a depth first fashion (emulating minimax), breath first fashion, or Monte Carlo Tree Search can be use to stochastically sample possible chains of random user and DM moves. Chains are then scored and each DM action is given the score of the average sub-tree that results from that action.

Scoring can be based on multiple criteria, some examples are below:

- Whether the user "wins" the game
- Location flow: whether the user moves around a lot
- Thought flow: whether the user pursues thematically similar actions or interleaves very different actions (for example pursuing one clue at a time or pursuing multiple clues simultaneously), as determined by plot graph connectivity.
- Suspense or other qualitative analysis: just because the story generator can tell a story in a suspenseful way doesn't mean that interactive players will adhere to the specific event ordering that would lead to suspense. Thus, causers and deniers may be useful in preserving the suspenseful nature of a story. The same applies to other criteria of "goodness".

Notes:

- This template is best suited for stories that are not highly linear, in that not all events are causally or temporally dependent on each other, and that the user can pursue different events in different orders without breaking the story. For example, there are multiple clues that can be investigated independently or in parallel. If a story is highly linear, meaning every event must happen in a precise order for the outcome, then this template will not provide any opportunities to do the search process. However, even though the story generator likely outputs a linear order of events, it there may be enough intermediate knowledge about the story to break up the story into a plot graph.

Source material:

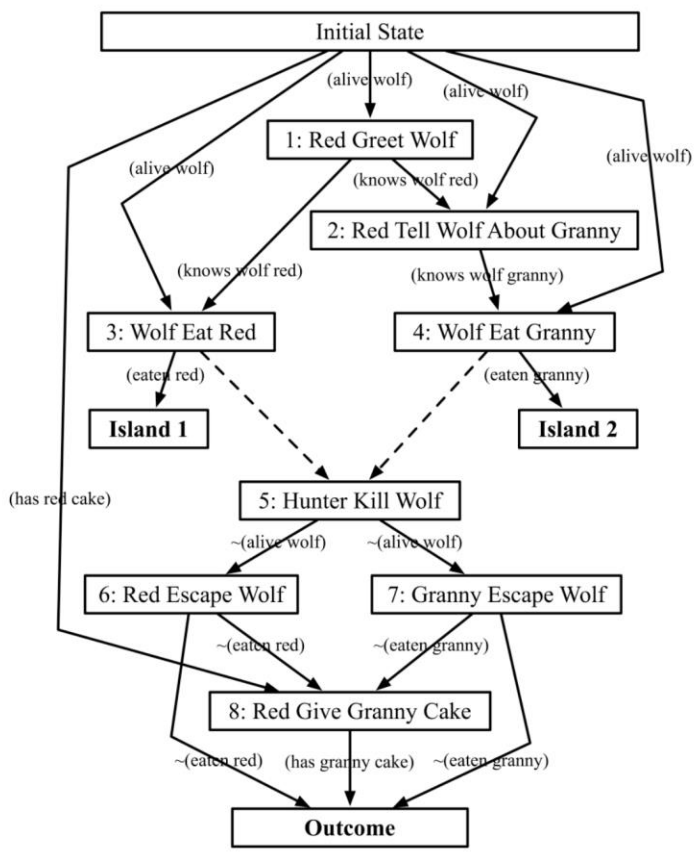
- Search-Based Drama Management in the Interactive Fiction Anchorhead, by Nelson and Mateas.

Template specific questions:

- In the original formulation of search based drama management, the game engine had a strong set of rules so that it was impossible to do anything other than perform actions that executed plot points or perform inconsequential actions such as movement. If your game engine allows arbitrary actions it is possible to perform actions that are exceptions, meaning that the eliminate all possibility of progressing to the end of the story. The search algorithms suggested above cannot handle infinite user actions. How will you recognize exceptions and how will you handle them?

2. Intervention and Accommodation

If a story can be expressed as a partially ordered plan with causal link between events, then these causal links can be used to indicate spans of time in which some condition in the world must remain true for the story to execute to completion. For example: in the partially ordered plan below, the solid arrows are causal links (dashed arrows are temporal ordering constraints), and Little Red Riding hood must have the cake for a certain span of time that encapsulates many other actions. Likewise, the wolf must be alive for a certain span, after which the status of the wolf is no longer important.



User actions can be classified as constituent (part of the story), consistent (not part of the story but do not negate any active causal links), or exceptional (not part of the story and negates active causal links). Exceptions must be handled via a process called accommodation, where the story plan is revised to account for the world state change. Such a system should keep track of the current world state as the plan executes, updating the current state and removing events that are executed. If an exception occurs, the affected parts of the plan that are no longer executable should be removed, and new parts of the plan should be generated to repair any events affected by the removal.

Notes:

- This template is best suited for story generators that already generate causal information for plans or reader models.
- It is possible to generate causal relationships for non-planning based generators by reasoning about the preconditions and effects of events and linking them together.
- The plan that is executing is probably the detective story, as opposed to the construction of how the original crime occurred before the detective became involved. Presumably the original crime does not change, but it is possible to change the details of the crime that have not been revealed.
- Event preconditions can be used to determine what state variables characters and objects should have, and the rules for action execution.

Source materials:

- Managing interaction between users and agents in a multi-agent storytelling environment, by Riedl, Saretto, and Young.
- Dynamic Experience Management in Virtual Worlds for Entertainment , Education , and Training, by Riedl and Stern.

Template specific questions:

- To original work on accommodation assumed a fixed set of user actions with known effects. When the user can enter any action, how can you recognize exceptions? The user may come up with actions that have effects that do not violate an active causal span, but commonsense would dictate that the event was rendered impossible. For example, a non-player character must perform “open door” which has causal links pre-identified for the character being in the same place as the door, and the character having a key. But the user performs “jam door with chair” which neither threatens the necessity of the other character being in the correct location nor the necessity of the character having the key. The problem is that there was no causal link (or state variable) pertaining to door being jammed or not.

3. Game Engine Rule Generation

This template considers how to handle user actions in the game world that are consistent with the story—that is, the actions themselves do not interfere with the progression of the story. In any open-ended interactive storytelling environment, the user does not know how the story will proceed. If they are trying to figure out what to do or how to proceed with solving a mystery, they will likely try many things that will not move the story forward. They may also try things that cannot be anticipated in advance by developers that do move the story forward. For example, if the user must break into a castle, the story may be for them to find a secret tunnel. However, the user might decide to try to try to scale the wall. If the developer of the game engine hasn't anticipated this in advance the game engine might simply respond with "I don't understand" or "you cannot do that". If the game engine is using an LLM to allow anything the user wants, the action may succeed but violate commonsense (how did you scale the wall? Did you have a rope or a ladder? Did you fly? Where did you get the rope or ladder? Does the rope or ladder even exist in the world?).

One way to resolve this issue is to let the user do anything they want, but only if they meet commonsense requirements to do this. For example, if the user wants to scale the wall they should be told they need something to climb with. If such a thing does not exist, it would then be created and added somewhere sensible in the world. Thus, the action is permitted, but the user has to work for it. The game engine should recognize that the user wants to perform an action and reason about the effects but also the preconditions. In essence, the game engine will just-in-time generate a new action for the user and once it is executed the game engine state will be updated accordingly. If the conditions for the action require new objects, new locations, or both, the world should be updated accordingly. These additions to the world may result in cascading changes where the user must now think of new actions to perform to meet those conditions. Some of these actions might already exist (move, pick up), but others might not, resulting in recursive creation of new rules and new world elements. For example: a ladder is created to meet the conditions for scaling a wall, but a ladder is too heavy to be carried, so the user is going to have to solve that problem by proposing new actions that don't exist.

When creating new actions/ rules, new state variables/slots may also need to be created. For example, once we have a ladder, the ladder might have a slot that indicates what it is leaning against. However, we might also need new slots on other elements in the world indicating what is scalable with a ladder, and what a ladder can be leaned against. More fundamentally, once we have new slots, we need to determine if existing actions/rules need to change. Do other actions change the leaning or leanability state information? Are

there actions, like “pick up” that need to applicability condition checks because they cannot be performed if the ladder is in the leaning state? Your solution should account for this new knowledge by retroactively updating how old actions/rules work.

Notes:

- Essentially, this project is about letting users create their own emergent mini-quests or side-quests. The project should demonstrate cascading new actions in which the user must now solve several now puzzles to do what they initially wanted.
- The demonstration should also demonstrate how old actions/rules have been updated by showing that those actions work properly with new objects, or by exposing information about the data structures underlying the actions/rules.

Source Material:

- STORY2GAME: Generating (Almost) Everything in an Interactive Fiction Game, by Zhou et al.

Template specific questions:

- The core of the project is about making all user actions permissible without violating commonsense constraints. However, this sets up two related issues. The first is that user may find a way to make progress through the story by performing different actions than the story would have anticipated. How does your system know when the story is advancing in a way other than specified by the story generator? How does your system know when the user is about to perform an action that will break the story and how will you handle that (e.g., preventing the action, changing the story)?

Development Considerations

One challenge you will face during development is that your story generator will probably produce slightly (or significantly) different stories each time. This means world generation might also happen differently each time. You may find it useful to generate a story and then save the requisite data structures to a file that you can load instead of hitting your story generator each time. Replicability is helpful for development and debugging. When you have the game engine in a stable state, you can link to the story generator and test against varied inputs.

For the final video demonstration, you may also find it advantageous to pre-generate the initial story so you can figure out what you want to have the user to do in order to show off

your system properly. The final video demonstration can start with that pre-generated story. However, additional interactions with the AI and story generator must be live. For example, if you need to regenerate the story, then the story generator should be involved for real.

Video Creation

The most straightforward means of creating a video will be a slideshow with voice-over, using screen video and audio recording.

Proposal videos:

- Introduce the team and team name.
- Indicate the template
- Provide an overview of your story generator
- Present the proposed interactive system overview and/or system architecture
- Address potential pitfalls
- Address how you plan to address the template-specific questions

Final videos:

- Introduce the team and team name.
- Indicate the template. If your system has a name, give that as well.
- Provide an overview of your story generator
- Present the final system overview and/or system architecture
- Address pitfalls you encountered and how you addressed them
- Address the template-specific questions
- Provide a video of an actual run of the system. The run should show the drama manager AI agent reacting to something the user does and/or intervening in the execution of the story and/or in the execution of the game engine.

Videos do not have a required length. They should be as long as necessary to address the above points.

One challenge in demonstrating interactive storytelling systems is that a lot of what the drama manager is doing is happening behind the scenes. If the DM is introducing a causer or denier (template 1), the user may not have any perceptual indication. If the DM is changing the story (template 2), then the user may not have any awareness that something is different. If the DM is creating new actions, objects, and locations, then the only

indication of this is in the updated data structures inside the game engine. You will need to find a way to show off the changes happening in the background and at the data structure/representation level. This can be automatically visualized, but you may also have some basic logging that you can draw attention to during the videos. You may also interleave playthrough demonstration with powerpoint explanations with diagrams of what is happening behind the scenes, though you will probably also want logs so that viewers of the videos can verify the changes are happening when you introduce them.

Cross-Team Assessments

In cross-team assessments, your team will review the videos from three other teams (which ones will be assigned). For each other team assigned to review, you will:

- Summarize their system
- Describe any pitfalls they may encounter that they do not already address in their videos (proposal stage), or summarize pitfalls that they encountered and how they addressed them (final stage).
- Describe how they intend or did address the template-specific questions (proposal stage), or how they did address template-specific questions (final stage)
- Suggest <u>two</u> things from lecture that would improve their lecture (proposal stage), or would have hypothetically improved their final design (final stage)

Note: cross-team assessments are not used in the grading of teams' proposals or final deliverables. It is used only to assess the reporting team's ability to connect the projects to lecture materials.

Cross team assessments should be 500+ words per reviewed system, for a total of 1500+ words.